RELATÓRIO DE PREPARAÇÃO DOS DADOS PARA MODELAGEM

WineQT — Etapa 3: preparação dos dados | Transformações sem vazamento | Transição para a modelagem

Objetivo do documento: registrar os códigos utilizados, explicar a finalidade de cada etapa, apresentar os cálculos e resultados e consolidar as decisões que deixam a base WineQT pronta, de forma reproduzível e sem vazamento de dados, para o treinamento dos modelos de classificação. Código completo: **Etapa3_Preparacao.ipynb**.

Resumo executivo

A base original tem **1.143 registros**. Sem considerar a coluna Id, foram encontradas **125 cópias duplicadas**, removidas para evitar que a mesma amostra apareça no treino e no teste; restaram **1.018 registros**. Não há valores ausentes nem valores fisicamente inconsistentes. O alvo quality_class (1 se quality ≥ 7) tem 13,46% de alta qualidade. A divisão estratificada 80/20 gerou **814 amostras de treino** e **204 de teste**, com a mesma proporção de classes. Outliers foram mantidos, nenhuma variável nova foi criada e a padronização (StandardScaler) foi ajustada somente no treino.

1. Parte 1 — Preparação dos dados

1.1 Revisão dos achados da EDA e leitura da base

```
RANDOM_STATE = 42
TEST_SIZE    = 0.20
LIMIAR_ALTA  = 7

df_original = pd.read_csv("WineQT.csv")
df = df_original.copy()
df.shape
df.dtypes
df.describe().T
```

Para que serve: as constantes fixam a semente aleatória, o tamanho do teste e o limiar do alvo, garantindo que qualquer pessoa reproduza exatamente os mesmos resultados. `copy()` preserva a base original para comparações; `dtypes` e `describe()` confirmam tipos e escalas.

Resultado: 1.143 linhas e 13 colunas, todas numéricas (11 medições físico-químicas em float64, quality e Id em int64). Como não há variáveis categóricas, não é necessário encoding.

Tema	Achado da EDA	O que a preparação faz com isso
Estrutura	1.143 linhas, 13 colunas	Confirmar tipos e formato
Ausentes / duplicatas	0 ausentes e 0 duplicatas (contadas com o Id)	Reconferir ignorando o Id
Classes	Alta: 13,91% · Baixa/média: 86,09%	Divisão estratificada e estratégia para desbalanceamento
Correlações	alcohol (+0,485) e volatile acidity (-0,407) as mais fortes	Manter todas as 11 preditoras
Outliers (IQR)	Maiores taxas em residual sugar (9,62%) e chlorides (6,74%)	Manter registros, sem remoção automática

1.2 Definição da variável alvo

```
df["quality_class"] = (df["quality"] >= LIMIAR_ALTA).astype(int)
pd.crosstab(df["quality"], df["quality_class"], margins=True)
```

A primeira linha cria quality_class: notas ≥ 7 recebem 1 (alta qualidade) e notas < 7 recebem 0 (baixa/média). O `crosstab` confere que cada nota caiu em exatamente uma classe. A coluna quality **não** é usada como preditora, pois é a própria origem do alvo (vazamento direto).

Nota (quality)	Classe 0	Classe 1	Total
3	6	0	6
4	33	0	33
5	483	0	483
6	462	0	462

Nota (quality)	Classe 0	Classe 1	Total
7	0	143	143
8	0	16	16
Total	**984**	**159**	**1.143**

1.3 Ausentes, duplicatas e inconsistências

```
colunas_fisico_quimicas = [c for c in df_original.columns if c not in ("Id", "quality")]

df.isna().sum()
df.duplicated().sum()                                          # com Id
df.duplicated(subset=colunas_fisico_quimicas + ["quality"]).sum() # sem Id
(df.groupby(colunas_fisico_quimicas)["quality"].nunique() > 1).sum()  # rótulos conflitantes
```

Para que serve: `isna().sum()` conta ausências. A EDA contou duplicatas com todas as colunas, inclusive o Id; como o Id é diferente em toda linha, essa contagem nunca encontraria repetições. Por isso a verificação foi refeita com `subset=`, ignorando o Id. O `groupby` verifica se medições idênticas receberam notas diferentes (rótulos conflitantes).

Verificação	Resultado
Valores ausentes	0 em todas as colunas
Duplicatas considerando o Id	0
Duplicatas ignorando o Id (medições + nota)	**125**
Linhas envolvidas em algum grupo duplicado	239
Grupos com medições idênticas e notas diferentes	0
Cópias excedentes que são de alta qualidade	22

```
(df[colunas_fisico_quimicas] < 0).sum().sum()                    # valores negativos
(df["free sulfur dioxide"] > df["total sulfur dioxide"]).sum()   # SO2 livre > total
(~df["pH"].between(0, 14)).sum()
(~df["density"].between(0.98, 1.05)).sum()
(~df["alcohol"].between(5, 20)).sum()
```

As regras de domínio verificam valores fisicamente impossíveis. Todas retornaram **0 ocorrências**: não há medições negativas, o SO2 livre nunca supera o SO2 total e pH, densidade, álcool e nota estão em faixas plausíveis.

```
df = (df.drop_duplicates(subset=colunas_fisico_quimicas + ["quality"], keep="first")
        .reset_index(drop=True))
```

Decisão: remover as 125 cópias, mantendo a primeira ocorrência. Todas têm a mesma nota, então não trazem informação nova. Se mantidas, a divisão aleatória poderia colocar uma cópia no treino e outra no teste: o modelo seria avaliado em amostras que já viu, inflando as métricas. Resultado: 1.143 → **1.018 linhas**; a proporção de alta qualidade passou de 13,91% para **13,46%**.

1.4 Verificação da coluna Id

```
df["Id"].is_unique
df["Id"].is_monotonic_increasing
df["Id"].min(), df["Id"].max()
df["Id"].corr(df["quality"])
```

Resultado: o Id é único e crescente, vai de 0 a 1597 com 455 lacunas (é a numeração de uma base maior da qual o WineQT é um recorte) e tem correlação de ≈ 0,08 com quality. **Decisão:** excluir o Id das preditoras. Ele não é uma medição do vinho; a correlação fraca reflete apenas a ordem de registro no arquivo, e usá-lo faria o modelo aprender um artefato. O Id é guardado à parte apenas para rastreabilidade.

1.5 Separação de X e y e divisão treino/teste

```
FEATURES = colunas_fisico_quimicas      # 11 medições físico-químicas
X = df[FEATURES]
y = df["quality_class"]

X_train, X_test, y_train, y_test, ids_train, ids_test = train_test_split(
    X, y, df["Id"], test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE)

pd.merge(X_train, X_test, how="inner", on=FEATURES)   # sobreposição treino/teste
```

Para que serve: X recebe apenas as 11 variáveis físico-químicas; y recebe o alvo binário. `train_test_split` separa 80% para treino e 20% para teste. `stratify=y` preserva a proporção da classe minoritária nos dois conjuntos, e `random_state=42` torna a divisão reproduzível. O `merge` confirma que nenhuma amostra idêntica aparece nos dois conjuntos. A divisão é feita **antes** de qualquer transformação, isolando o teste.

Conjunto	Amostras	Alta qualidade (1)	% alta qualidade
Base limpa	1.018	137	13,46%
Treino	814	110	13,51%
Teste	204	27	13,24%

Amostras idênticas entre treino e teste: **0**.

2. Parte 2 — Transformações sem vazamento de dados

2.1 Outliers: sinalizados, não removidos

```
q1, q3 = X_train.quantile(0.25), X_train.quantile(0.75)
iqr = q3 - q1
fora = X_train.lt(q1 - 1.5 * iqr) | X_train.gt(q3 + 1.5 * iqr)
fora.sum(); fora.mean() * 100
y_train[fora[coluna]].mean() * 100    # % de alta qualidade entre os sinalizados
```

Os limites do IQR foram recalculados **somente com o treino**. A última linha mede, entre as amostras sinalizadas em cada variável, quantas são de alta qualidade, para avaliar se os extremos carregam informação.

Variável	Qtd. (treino)	% treino	% alta qualidade entre sinalizados
residual sugar	66	8,11%	25,76%
chlorides	52	6,39%	7,69%
sulphates	35	4,30%	17,14%
fixed acidity	28	3,44%	25,00%
total sulfur dioxide	28	3,44%	7,14%
density	25	3,07%	36,00%
pH	16	1,97%	25,00%
free sulfur dioxide	9	1,11%	11,11%
volatile acidity	8	0,98%	0,00%
alcohol	5	0,61%	40,00%
citric acid	1	0,12%	0,00%

Decisão: manter todos os outliers. São valores fisicamente plausíveis (passaram nas regras de domínio); 197 amostras de treino (24,2%) têm ao menos uma sinalização, e excluí-las descartaria um quarto da base. Além disso, entre os sinalizados em density, residual sugar, fixed acidity, pH e alcohol, a proporção de vinhos de alta qualidade (25-40%) é **maior** que a média do treino (≈ 13,5%): os extremos ajudam a identificar justamente a classe minoritária.

2.2 Necessidade de padronização por modelo

As variáveis estão em escalas muito diferentes: density varia na 3ª casa decimal (desvio 0,0019), enquanto total sulfur dioxide vai até 289 (desvio 33,8). O efeito disso depende do tipo de modelo:

Família de modelo	Precisa de escala?	Justificativa
Regressão Logística	Sim	A regularização penaliza os coeficientes por igual; sem escala, variáveis com números grandes são tratadas de forma injusta.
KNN	Sim	Usa distâncias; a variável de maior escala dominaria o cálculo.
SVM	Sim	Margem e kernel dependem de distâncias.
Árvore, Random Forest, Gradient Boosting	Não	Dividem os dados por limiares; a ordem dos valores não muda com a escala.

Por isso a Etapa 4 recebe **duas versões** dos dados: a original (modelos de árvore) e a padronizada (modelos baseados em distância ou coeficientes).

2.3 Ajuste do StandardScaler somente no treino

```
scaler = StandardScaler()
scaler.fit(X_train)                      # aprende média e desvio APENAS do treino
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test) # aplica os parâmetros já ajustados
```

Explicação: `fit` calcula a média e o desvio padrão de cada variável usando apenas o treino. `transform` aplica (x - média) / desvio com esses parâmetros fixos aos dois conjuntos; o teste nunca é usado para ajustar nada. O StandardScaler foi escolhido em vez do Min-Max porque o Min-Max depende do mínimo e do máximo — justamente os extremos que decidimos manter — e um único outlier comprimiria o restante da variável.

Variável	Média (treino)	Desvio (treino)	Média do teste após transformação
fixed acidity	8,3269	1,7396	-0,110
volatile acidity	0,5315	0,1852	0,054
citric acid	0,2750	0,1978	-0,158
residual sugar	2,4699	1,1919	0,228
chlorides	0,0885	0,0518	-0,124
free sulfur dioxide	15,5627	10,1220	0,042
total sulfur dioxide	46,1744	33,8030	0,022
density	0,9967	0,0019	-0,034
pH	3,3097	0,1600	0,056
sulphates	0,6628	0,1751	-0,172
alcohol	10,4442	1,0797	0,038

Leitura: no treino, a média ficou 0 e o desvio 1 em todas as variáveis, por construção. No teste os valores ficam próximos, mas **não exatamente** 0 — sinal de que o teste não participou do ajuste.

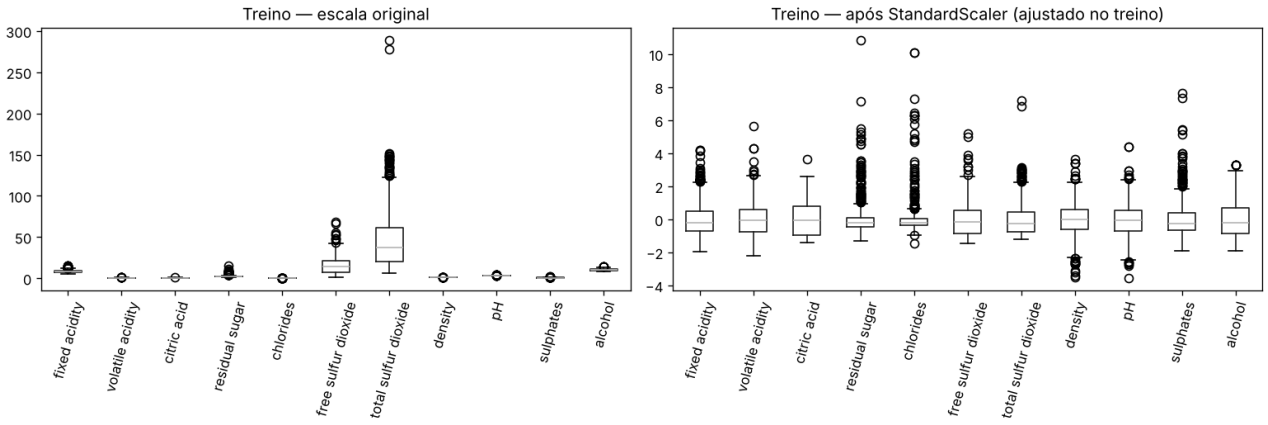


Figura 1 — Variáveis do treino antes (esq.) e depois (dir.) da padronização. Gerada em dados_preparados/antes_depois_padronizacao.png.

Assimetria: chlorides (5,68), residual sugar (4,17) e sulphates (2,49) têm cauda longa à direita. Uma transformação logarítmica **não** foi aplicada por não haver evidência de que seja necessária; fica registrada como teste possível na Etapa 4 caso a Regressão Logística tenha desempenho fraco.

2.4 Avaliação de feature engineering

```
cand["total_acidity"] = X_train["fixed acidity"] + X_train["volatile acidity"] + X_train["citric acid"]
cand["bound_so2"]     = X_train["total sulfur dioxide"] - X_train["free sulfur dioxide"]
cand["so2_ratio"]     = X_train["free sulfur dioxide"] / X_train["total sulfur dioxide"]
cand.corrwith(y_train)                                  # associação com o alvo
[X_train.corrwith(cand[c]).abs().max() for c in cand]      # redundância
```

Critério: a variável nova só entra se tiver associação com o alvo **e** não for redundante com uma variável já existente. A análise usa somente o treino.

Candidata	Corr. com alvo	Variável mais parecida	Corr. com ela	Decisão
total_acidity	0,095	fixed acidity	0,996	Não criar — redundante
bound_so2	-0,112	total sulfur dioxide	0,963	Não criar — redundante
so2_ratio	0,130	total sulfur dioxide	0,384	Não criar — candidata opcional na Etapa 4

Para comparação, as maiores associações das variáveis originais com o alvo no treino são alcohol (0,417), volatile acidity (0,299), citric acid (0,224), sulphates (0,192) e density (0,174). A so2_ratio fica abaixo de todas e só deve ser incorporada se melhorar as métricas na validação cruzada do treino.

2.5 Desbalanceamento de classes

Cerca de 13,5% das amostras são de alta qualidade. Um modelo que sempre respondesse “baixa/média” teria ≈ 86,5% de acurácia sem identificar nenhum vinho bom. **Nenhuma reamostragem foi aplicada nesta etapa**, para não alterar o teste nem antecipar decisões de modelagem. Recomendações para a Etapa 4: usar class_weight="balanced"; validação com StratifiedKFold; métricas de precisão, recall e F1 da classe 1, ROC-AUC, PR-AUC e matriz de confusão; se houver SMOTE, aplicá-lo somente dentro do treino, em um Pipeline.

2.6 Arquivos gerados

```
X_train.to_csv("dados_preparados/X_train.csv", index=False)
X_train_scaled.to_csv("dados_preparados/X_train_scaled.csv", index=False)
joblib.dump(scaler, "dados_preparados/scaler_treino.joblib")
json.dump(registro, open("dados_preparados/registro_preparacao.json", "w"))
```

Arquivo gerado	O que contém
X_train.csv / X_test.csv	Preditoras na escala original (modelos de árvore).
X_train_scaled.csv / X_test_scaled.csv	Preditoras padronizadas com o scaler do treino (LogReg, KNN, SVM).
y_train.csv / y_test.csv	Alvo binário quality_class.
scaler_treino.joblib	StandardScaler ajustado no treino, para reutilização.
ids_divisao.csv	Id de cada amostra e o conjunto em que caiu (rastreabilidade).
wineqt_limpo.csv	Base sem duplicatas, com quality_class.
registro_preparacao.json	Parâmetros e decisões desta etapa.
antes_depois_padronizacao.png	Boxplots do treino antes e depois da padronização.

Todos os arquivos ficam na pasta dados_preparados. O próprio notebook recarrega os arquivos e confere que eles reproduzem exatamente os dados preparados. Para validação cruzada na Etapa 4, recomenda-se usar os dados originais dentro de um Pipeline(StandardScaler, modelo), para que a padronização seja reajustada em cada dobra.

3. Decisões finais

Tema	Decisão
Variável alvo	quality_class = 1 se quality ≥ 7; 0 se quality < 7.
Valores ausentes	Nenhum encontrado; nenhuma imputação.
Inconsistências	Nenhuma encontrada nas regras de domínio; nenhum valor corrigido.
Duplicatas	125 cópias removidas (1.143 → 1.018), por representarem a mesma amostra e causarem vazamento entre treino e teste.
Coluna Id	Excluída das preditoras: identificador de linha, sem significado físico-químico.
Coluna quality	Excluída das preditoras: origem do alvo.
Divisão treino/teste	80/20, estratificada, random_state=42 (814 / 204 amostras).
Outliers	Mantidos; IQR recalculado no treino apenas para registro.
Padronização	StandardScaler ajustado só no treino; versões original e padronizada disponíveis.
Feature engineering	Nenhuma variável criada; so2_ratio registrada como teste opcional.
Desbalanceamento	Sem reamostragem; tratar na modelagem com class_weight, StratifiedKFold e métricas da classe 1.

4. Transição entre EDA, preparação e modelagem

Etapa	Responsabilidade
EDA (Etapa 2)	Entrega os achados sobre estrutura, distribuição, correlações, classes e possíveis outliers.
Preparação (Etapa 3)	Confere os achados, define o alvo binário, remove duplicatas, organiza X e y, faz a divisão treino/teste e ajusta a padronização sem vazamento.
Desenvolvimento de ML (Etapa 4)	Carrega dados_preparados, treina pelo menos dois classificadores (sugestão: Regressão Logística com dados padronizados e Random Forest com dados originais) e gera previsões.
Avaliação (Etapa 5)	Compara os modelos no teste com precisão, recall, F1, ROC-AUC/PR-AUC e matrizes de confusão, e interpreta os resultados.

Conclusão para apresentação

A preparação dos dados conferiu os achados da EDA e deixou a base pronta para a modelagem. A principal descoberta foi a existência de 125 registros repetidos, que não apareciam na EDA porque a contagem incluía o identificador; eles foram removidos para que o modelo não seja avaliado em vinhos que já viu. O identificador e a nota original foram retirados das variáveis de entrada, e a base foi dividida em treino e teste preservando a proporção de vinhos de alta qualidade. Os valores extremos foram mantidos, pois são plausíveis e concentram vinhos de alta qualidade. A padronização foi aprendida somente com o treino, mantendo o teste isolado para uma avaliação justa.

Nota metodológica: as correlações usadas na avaliação de feature engineering medem apenas associação linear com o alvo binário e não indicam causalidade. A decisão final sobre so2_ratio e transformações logarítmicas deve ser tomada com validação cruzada no treino, nunca com o conjunto de teste.